Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

ОТЧЁТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Тип практики: Преддипломная практика

Название предприятия: НУК ИУ МГТУ им. Н.Э.Баумана

Студент: группа ИУ8-124-2026 л.д. 20У474
Железцов Никита Владимирович _____ (подпись, дата)

Руководитель от предприятия:
доцент кафедры ИУ8 Медведев Николай Викторович _____ (подпись, дата)

Руководитель ВКР:
доцент кафедры ИУ8 Медведев Николай Викторович _____ (подпись, дата)

Руководитель от кафедры:
доцент кафедры ИУ8 Зайцева Анастасия Владленовна _____ (подпись, дата)

Оценка: _____

Москва, 2026

РЕФЕРАТ

Отчёт содержит 33 стр., 7 рис., 5 источн., 1 прил.

Работа посвящена верификации соответствия трасс исполнения программы формальной спецификации на языке TLA+.

Целью работы является разработка методики сопоставления трассы исполнения программы с формальной спецификацией, формализация соответствующей математической постановки задачи и обоснование её решения.

В работе предложена методика, основанная на инструментировании программной реализации, построении трассы наблюдаемых шагов исполнения и её интерпретации как системы ограничений на переходы формальной модели. Сформулирована математическая постановка задачи в терминах системы переходов, множества допустимых поведений модели и множества поведений, согласованных с трассой. Показано, что рассматриваемая задача является задачей разрешимости. Для её решения введена последовательность множеств достижимых состояний, согласованных с префиксами трассы, и на этой основе получен алгоритм проверки согласованности трассы с моделью.

Практическая реализация предложенного подхода выполнена средствами TLA+ и TLC на примере спецификации протокола двухфазной фиксации транзакции.

Ключевые слова: TLA+, TLC, формальная верификация, система переходов, трасса исполнения, распределённые системы.

СОДЕРЖАНИЕ

РЕФЕРАТ . . . 3
ВВЕДЕНИЕ . . . 5
ОСНОВНАЯ ЧАСТЬ . . . 7
1 Характеристика организации . . . 8
2 Методика верификации . . . 9
3 Математическая постановка задачи . . . 10
3.1 Формализация модели и ее поведения . . . 10
3.2 Формализация трассы программы . . . 12
3.3 Согласованность трассы с моделью . . . 12
3.4 Итоговая математическая постановка задачи . . . 15
4 Обоснование решения математической постановки задачи . . . 16
4.1 Основная идея решения . . . 16
4.2 Рекуррентное построение множеств D_i . . . 16
4.3 Доказательство построения множеств D_i . . . 18
4.4 Критерий существования согласованного поведения . . . 19
4.5 Алгоритм решения . . . 20
5 Реализация задачи и решения средствами TLA+ и TLC . . . 22
5.1 Общая схема реализации . . . 22
5.2 Соответствие модели и TLA+ спецификации . . . 22
5.3 Представление трассы программы . . . 24
5.4 Реализация предиката согласованности шага . . . 25
5.5 Ограниченная спецификация как реализация $\mathrm{M}(\tau)$ . . . 27
5.6 Соответствие рекуррентных множеств D_i и работы TLC . . . 28
5.7 Критерий успешной проверки в TLC . . . 29
5.8 Особенности и ограничения рассматриваемой реализации . . . 30
ЗАКЛЮЧЕНИЕ . . . 31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 32
ПРИЛОЖЕНИЕ А Методика верификации . . . 33

ВВЕДЕНИЕ

Формальные спецификации применяются для описания допустимого поведения сложных программных систем, в особенности распределённых алгоритмов, для которых характерны недетерминизм, конкурентное взаимодействие и большое число возможных сценариев выполнения. Одним из распространённых средств такого описания является язык TLA+, позволяющий задавать систему в терминах состояний и переходов, а также проверять свойства модели средствами модельной проверки.

Однако наличие формальной спецификации само по себе не гарантирует её актуальность по отношению к реальной реализации. В процессе разработки исходный код системы изменяется, дополняется новыми возможностями и оптимизируется, тогда как формальная модель нередко обновляется с запозданием или не обновляется вовсе. В результате возникает риск расхождения между фактическим поведением программы и поведением, допускаемым спецификацией. Подобное расхождение снижает ценность формальной модели как инструмента анализа и верификации.

Одним из возможных способов решения данной проблемы является валидация трасс исполнения программы относительно формальной спецификации. В этом подходе выполнение программы наблюдается и фиксируется в виде трассы, содержащей сведения о происходящих событиях и изменениях логических переменных. Затем эта трасса интерпретируется в терминах формальной модели, после чего проверяется, существует ли допустимое поведение спецификации, согласованное с наблюдаемым исполнением. Такой подход позволяет сравнивать реализацию и спецификацию без необходимости полного доказательства корректности программы относительно модели.

Актуальность работы определяется необходимостью построения формального и практически реализуемого метода проверки согласованности между исходным кодом и TLA+-спецификацией. Особенно важна такая проверка для распределённых систем, где вручную сопоставить поведение реализации и модели затруднительно из-за сложности пространства состояний и большого числа возможных межпроцессных взаимодействий.

Целью работы является разработка методики верификации соответствия трасс исполнения программы и формальной спецификации на языке TLA+, а также формализация математической постановки этой задачи и обоснование её решения.

Место прохождения практики – Научно-учебный комплекс «Информатика и системы управления» МГТУ им. Н.Э. Баумана. Период прохождения практики – с 9 февраля 2026 года по 10 марта 2026 года.

В задачи прохождения практики входило:

а) Сформулировать математическую постановку задачи верификации соответствия TLA+ спецификации и исходного кода;

б) Предложить возможные методы решения поставленной задачи.

Практическая значимость работы заключается в том, что предложенный подход может использоваться для автоматизированного выявления расхождений между реализацией и формальной моделью, а также для локализации шага трассы, на котором впервые возникает несоответствие.

Структурно работа состоит из разделов, посвящённых описанию методики верификации, математической постановке задачи, обоснованию её решения и реализации полученных результатов средствами TLA+ и TLC.

ОСНОВНАЯ ЧАСТЬ

В основной части приводится краткая характеристика организации, в которой проходила практика, описывается предлагаемая методика верификации, формулируется математическая постановка задачи проверки согласованности трассы программы с формальной спецификацией, обосновывается её решение и рассматривается реализация предложенного подхода средствами TLA+ и TLC.

1 Характеристика организации

Научно-учебный комплекс «Информатика и системы управления» [1] (далее — НУК ИУ) является самостоятельным структурным подразделением МГТУ им. Н.Э. Баумана (далее — Университет). Полное наименование НУК ИУ на английском языке Scientific Educational Complex «Informatics and Control Systems» Bauman Moscow State Technical University. Сокращённое — SEC ICS BMSTU.

НУК ИУ возглавляет руководитель НУК ИУ д.т.н., профессор и заведующий кафедрой ИУ6 («Компьютерные системы и сети») Пролетарский А.В., который непосредственно подчиняется действующему ректору Университета к.т.н. Гордину М.В..

Цель деятельности НУК ИУ: обеспечение образовательного и научного процессов в соответствии с Уставом Университета. К основным задачам подразделения относят:

– Подготовку бакалавров, специалистов, магистров, научных и научно-педагогических кадров высшей квалификации в соответствии с государственными лицензиями, выданными Университету.

– Выполнение фундаментальных, поисковых и прикладных научных исследований, проведение опытно-конструкторских работ и производство опытных образцов перспективной техники и технологий.

– Написание и издание учебников, учебных пособий и монографий.

– Развитие научных, педагогических и инженерных школ НУК ИУ.

– Разработку и внедрение прогрессивных форм, методов и средств подготовки специалистов.

– Повышение квалификации и переподготовка научно-педагогических кадров технических и экономических учебных заведений, а также специалистов предприятий с высшим образованием.

– Развитие материально-технической базы НУК ИУ.

– Организацию научной работы студентов.

– Социальную поддержку и защиту преподавателей, научного и отвечающих требованиям подготовки вспомогательного персонала.

2 Методика верификации

Предлагаемая методика верификации основана на сопоставлении трасс исполнения программы с формальной спецификацией, написанной на языке TLA+. Схема подхода представлена в Приложении А.

Исходными артефактами являются программная реализация алгоритма и формальная спецификация этого алгоритма на языке TLA+. Спецификация задаёт множество допустимых состояний системы и переходов между ними, однако не содержит сведений о конкретной реализации. Реализация, в свою очередь, выполняется в распределённой среде, где возможны различные порядки доставки сообщений и недетерминированные сценарии выполнения.

Для сопоставления поведения реализации со спецификацией используется инструментирование кода. В реализацию внедряется библиотека трассировки, которая фиксирует события, соответствующие переходам спецификации, а также изменения переменных спецификации. При выполнении системы каждый процесс (координатор и участники) формирует собственную трассу в формате JSON, содержащую последовательность операций обновления логических переменных модели. Полученные локальные трассы агрегируются в единый глобальный файл, отражающий наблюдаемую последовательность изменений состояния всей распределённой системы. Таким образом, формируется абстрактное представление исполнения, выраженное в терминах переменных TLA+, а не в терминах низкоуровневых операций кода.

Для проверки корректности трассы используется дополнительная спецификация, построенная на основе исходной TLA+ спецификации. Она расширяет её таким образом, чтобы поведение модели было ограничено наблюдаемой последовательностью изменений. Иными словами, трасса интерпретируется как частичное ограничение на допустимые переходы модели.

Далее средство модельной проверки TLC используется для проверки существования поведения спецификации, согласованного с данной трассой. Если TLC подтверждает корректность, это означает, что наблюдаемое исполнение допустимо спецификацией. В противном случае обнаруживается расхождение между реализацией и моделью, и TLC предоставляет информацию, позволяющую локализовать проблему.

3 Математическая постановка задачи

В данном разделе задача верификации соответствия формальной спецификации и исходного кода формализуется в терминах системы переходов, трассы наблюдений и предиката согласованности шага. На этой основе задаётся строгий критерий согласованности трассы с моделью как условие существования допустимого поведения модели, совместимого с наблюдаемой трассой.

3.1 Формализация модели и ее поведения

Для формализации модели (спецификации) программы используется понятие *системы переходов* (transition system) [2] с выделенным множеством начальных состояний.

Определение 3.1. Системой переходов называется тройка

$$\mathcal{M} = (X, X_0, R), \tag{1}$$

где

– X — множество состояний;
– $X_0 \subseteq X$ — множество начальных состояний;
– $R \subseteq X \times X$ — отношение переходов.

Элемент $x \in X$ интерпретируется как полное абстрактное описание состояния системы в некоторый момент её выполнения. Условие $(x, y) \in R$ означает, что из состояния x система может перейти в состояние y за один допустимый шаг. Иначе говоря, отношение R задаёт все допустимые одношаговые изменения состояния системы.

Следовательно, система переходов $\mathcal{M}$ задаёт не одно конкретное выполнение, а всё множество возможных выполнений модели. Именно это множество допустимых выполнений и используется далее как эталонное описание поведения системы.

Определение 3.2. Конечным поведением системы переходов $\mathcal{M}$ длины $n \in \mathbb{N}_0$ называется последовательность состояний

$$\pi_n = (x_0, x_1, \ldots, x_n) \in X^{n+1}, \tag{2}$$

для которой выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R.$$

Первое условие означает, что поведение начинается из некоторого допустимого начального состояния. Второе условие означает, что каждый следующий элемент последовательности получается из предыдущего допустимым переходом системы. Таким образом, конечное поведение есть путь в ориентированном графе, порождаемом системой переходов, начинающийся в одной из вершин из множества X_0.

Определение 3.3. Для каждого $n \in \mathbb{N}_0$ обозначим через $\mathrm{B}_n(\mathcal{M})$ множество всех конечных поведений системы переходов $\mathcal{M}$ длины n:

$$\mathrm{B}_n(\mathcal{M}) = \left\{(x_0, \ldots, x_n) \in X^{n+1} : x_0 \in X_0,\ \forall i \in \{0, \ldots, n-1\} : (x_i, x_{i+1}) \in R\right\}.$$

Тогда множество всех конечных поведений системы переходов $\mathcal{M}$ обозначается через $\mathrm{B}(\mathcal{M})$ и определяется равенством

$$\mathrm{B}(\mathcal{M}) = \bigcup_{n \in \mathbb{N}_0} \mathrm{B}_n(\mathcal{M}). \tag{3}$$

Множество $\mathrm{B}(\mathcal{M})$ содержит все конечные пути в системе переходов, начинающиеся из допустимых начальных состояний. Тем самым оно полностью определяет допустимое наблюдаемое поведение модели на конечных префиксах.

Если некоторое исполнение реализации не может быть согласовано ни с одним элементом множества $\mathrm{B}(\mathcal{M})$, то это означает, что наблюдаемое поведение не принадлежит множеству допустимых поведений, задаваемых абстрактной моделью. Следовательно, в этом случае фиксируется расхождение между реализацией и спецификацией.

3.2 Формализация трассы программы

Реальная программа в процессе выполнения порождает наблюдаемую трассу. В рассматриваемой постановке предполагается, что трасса не содержит отдельного наблюдения начального состояния системы. Начальное состояние выбирается из множества начальных состояний модели, а трасса фиксирует только наблюдения, относящиеся к последующим шагам выполнения программы.

При этом в рамках рассматриваемой постановки предполагается, что каждой записи трассы соответствует ровно один переход поведения модели. Иначе говоря, трасса и поведение модели сопоставляются *пошагово*.

Определение 3.4. Пусть Σ — множество всех возможных наблюдений. Тогда наблюдаемая трасса длины $m \in \mathbb{N}_0$ задаётся последовательностью

$$\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m. \tag{4}$$

Каждый элемент σ_i интерпретируется как наблюдение, относящееся к i-му шагу выполнения программы. Таким образом, поведение модели задаётся последовательностью состояний $(x_0, x_1, \ldots, x_m)$, тогда как наблюдаемая трасса задаётся последовательностью наблюдений $(\sigma_1, \sigma_2, \ldots, \sigma_m)$, в которой каждый элемент σ_i относится к переходу из состояния x_{i-1} в x_i.

3.3 Согласованность трассы с моделью

После введения понятий модели и трассы можно строго определить, что означает их согласованность.

В общем случае трасса программы представляет собой последовательность неполных изменений состояния системы: обычно она содержит лишь частичную информацию о каждом шаге выполнения, существенную для верификации, например сведения о произошедшем событии и об изменениях отдельных компонент состояния. Поэтому необходимо формально задать, когда переход считается согласованным с данным наблюдением.

Определение 3.5. Предикатом согласованности наблюдаемого шага называется отображение

$$C : \Sigma \times X \times X \to \{0, 1\}. \tag{5}$$

Значение $C(\sigma, x, y) = 1$ означает, что наблюдение σ согласованно с переходом из состояния x в состояние y.

Предикат C задаёт способ интерпретации трассы в терминах абстрактной модели. Его конкретный вид определяется составом данных, записываемых в трассу. Например, если наблюдение фиксирует имя произошедшего события и значения некоторых изменяемых компонент состояния, то согласованность означает, что рассматриваемый переход принадлежит классу переходов, соответствующему данному событию, и приводит к зафиксированным изменениям состояния. Если же наблюдение фиксирует значения лишь некоторых компонент следующего состояния, то согласованность означает совпадение именно этих компонент, тогда как остальные компоненты могут выбираться произвольно при условии допустимости перехода.

Таким образом, трасса задаёт не одно конкретное поведение системы, а систему ограничений на каждый последующий переход.

Определение 3.6. Пусть дана трасса $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$. Тогда поведение модели $\pi = (x_0, x_1, \ldots, x_m) \in X^{m+1}$ называется *согласованным с трассой* τ, если выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1.$$

Иначе говоря, трасса длины m задаёт ограничения на поведение модели, состоящее из m переходов и $m + 1$ состояний. В рамках данной постановки не рассматриваются сопоставления, при которых одному элементу трассы соответствует несколько переходов модели или, наоборот, одному переходу модели соответствует несколько элементов трассы. Такое ограничение соответствует используемой схеме проверки, в которой каждая запись трассы обрабатывается за один шаг ограниченной спецификации.

Определение 3.7. Множество всех последовательностей состояний (поведений), согласованных с трассой наблюдений $\tau = (\sigma_1, \ldots, \sigma_m)$, обозначается через $\mathrm{M}(\tau)$ и определяется равенством

$$\mathrm{M}(\tau) = \left\{(x_0, \ldots, x_m) \in X^{m+1} : x_0 \in X_0,\ \forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1\right\}. \tag{6}$$

Важно подчеркнуть, что элементы множества $\mathrm{M}(\tau)$ сами по себе ещё не обязаны быть допустимыми поведениями модели. В этом множестве учитываются только ограничения, задаваемые трассой. Чтобы последовательность состояний представляла собой корректное абстрактное поведение системы, она дополнительно должна принадлежать множеству $\mathrm{B}(\mathcal{M})$.

Следовательно, трасса является согласованной с моделью тогда и только тогда, когда существует хотя бы одно поведение, которое одновременно:

- является допустимым поведением системы переходов;
- согласовано с наблюдаемой трассой.

Это приводит к следующему строгому определению.

Определение 3.8. Трасса τ называется *согласованной* с моделью $\mathcal{M}$, если

$$\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) \neq \varnothing. \tag{7}$$

Если же $\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) = \varnothing$, то ни одно допустимое поведение модели не может объяснить наблюдаемую трассу, и, следовательно, фиксируется расхождение между реализацией и абстрактной моделью.

При этом в данной постановке нельзя требовать выполнения включения $\mathrm{M}(\tau) \subseteq \mathrm{B}(\mathcal{M})$. Действительно, множество $\mathrm{M}(\tau)$ определяется только наблюдаемой трассой и, вообще говоря, строится по неполной информации о выполнении программы. Поэтому оно содержит все последовательности состояний, согласованные с зафиксированными наблюдениями, включая и такие последовательности, которые удовлетворяют этим наблюдениям, но не являются допустимыми поведениями модели. Иными словами, из согласованности последовательности с трассой ещё не следует, что эта последовательность принадлежит множеству $\mathrm{B}(\mathcal{M})$.

3.4 Итоговая математическая постановка задачи

Пусть заданы:

- модель - система переходов $\mathcal{M} = (X, X_0, R)$, где X — множество состояний, $X_0 \subseteq X$ — множество начальных состояний, $R \subseteq X \times X$ — отношение переходов;
- трасса программы $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$, где Σ - множество наблюдений
- предикат согласованности шага $C : \Sigma \times X \times X \to \{0, 1\}$.

Тогда задача верификации соответствия трассы программы и модели состоит в том, чтобы определить, существует ли последовательность состояний $(x_0, x_1, \ldots, x_m) \in X^{m+1}$, для которой выполнены условия $x_0 \in X_0, \forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1$.

Иными словами, требуется установить, существует ли хотя бы одно допустимое поведение модели, пошагово согласованное с наблюдаемой трассой программы:

$$\left\{(x_0, \ldots, x_m) \in X^{m+1} : x_0 \in X_0,\ \forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1\right\}. \quad (8)$$

Эквивалентно, требуется установить непустоту пересечения множества , то есть проверить истинность условия

$$\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) \neq \varnothing, \quad (9)$$

где $\mathrm{B}(\mathcal{M})$ - множество всех конечных поведений системы переходов $\mathcal{M}$, а $\mathrm{M}(\tau)$ - множество всех конечных поведений, согласованных с трассой τ.

Следовательно, рассматриваемая задача является *задачей разрешимости* (decision problem): по заданным объектам $\mathcal{M}, \tau, C$ необходимо выдать бинарный ответ на вопрос, истинно или ложно утверждение (8).

4 Обоснование решения математической постановки задачи

В предыдущем разделе задача верификации соответствия трассы программы и формальной модели была сведена к проверке непустоты пересечения множеств $\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau)$, где $\mathrm{B}(\mathcal{M})$ — множество всех конечных поведений модели, а $\mathrm{M}(\tau)$ — множество всех конечных последовательностей состояний, согласованных с наблюдаемой трассой τ. В настоящем разделе предлагается строгое решение этой задачи и доказывается его корректность.

4.1 Основная идея решения

Из формулировки задачи непосредственно следует естественная идея решения. Поскольку трасса состоит из конечного числа наблюдений $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m)$, то проверку можно выполнять последовательно:

а) сначала определить множество состояний, достижимых после согласования первого наблюдения σ_1;

б) затем, исходя из уже найденного множества, определить множество состояний, достижимых после согласования первых двух наблюдений (σ_1, σ_2);

в) продолжать этот процесс вплоть до последнего наблюдения σ_m;

г) в конце проверить, осталось ли хотя бы одно достижимое состояние.

Если после обработки всех наблюдений хотя бы одно состояние остаётся достижимым, то существует поведение модели, согласованное с трассой. Если же после обработки некоторого шага множество достижимых состояний становится пустым, то никакое продолжение уже невозможно, а значит, трасса не согласована с моделью.

Тем самым задача проверки существования согласованного поведения сводится к последовательному вычислению достижимых множеств состояний.

4.2 Рекуррентное построение множеств D_i

Для формализации указанной идеи введём последовательность множеств $D_0, D_1, \ldots, D_m \subseteq X$, где каждое множество D_i интерпретируется как множество всех состояний модели, достижимых после согласования первых i наблюдений трассы.

Поскольку трасса не содержит отдельного наблюдения начального состояния, исходное множество определяется только множеством начальных состояний модели.

Определение 4.1. Начальное множество достижимых согласованных состояний определяется равенством

$$D_0 = X_0. \tag{10}$$

Это определение означает, что до обработки трассы допустимыми считаются все начальные состояния модели.

Далее, для каждого $i \in \{1, \ldots, m\}$ определим множество D_i рекуррентно.

Определение 4.2. Для каждого $i \in \{1, \ldots, m\}$ множество достижимых согласованных состояний после обработки первых i наблюдений определяется равенством

$$D_i = \{y \in X : \exists x \in D_{i-1} : C(\sigma_i, x, y) = 1\}. \tag{11}$$

Смысл этого определения состоит в следующем. Состояние y включается в множество D_i тогда и только тогда, когда существует хотя бы одно состояние $x \in D_{i-1}$, уже достижимое после согласования первых $i-1$ наблюдений, такое, что переход из x в y согласован с i-м наблюдением σ_i.

Определение 4.2 отражает принцип пошагового распространения ограничений. Действительно, каждое следующее наблюдение σ_i не рассматривается изолированно, а применяется только к тем состояниям, которые уже достижимы после обработки предыдущего префикса трассы.

Это означает, что:

- множество D_0 описывает все возможные исходные положения системы;
- множество D_1 описывает все состояния, в которые система может перейти после шага, согласованного с σ_1;
- множество D_2 описывает все состояния, в которые система может перейти после двух последовательных шагов, согласованных с (σ_1, σ_2);
- и так далее вплоть до множества D_m.

4.3 Доказательство построения множеств D_i

Для обоснования корректности предложенного решения необходимо доказать, что множество D_i действительно имеет именно тот смысл, который ему приписан, а именно: содержит в точности те состояния, которые достижимы после согласования первых i наблюдений трассы.

Утверждение 1. Для любого индекса $i \in \{0, \ldots, m\}$ и любого состояния $x \in X$ следующие утверждения эквивалентны:

а) $x \in D_i$;

б) существует последовательность состояний $(x_0, x_1, \ldots, x_i) \in X^{i+1}$, для которой выполнены условия

$$x_0 \in X_0, \qquad x_i = x, \qquad \forall k \in \{1, \ldots, i\} : C(\sigma_k, x_{k-1}, x_k) = 1.$$

Иначе говоря, состояние x принадлежит множеству D_i тогда и только тогда, когда существует путь длины i, начинающийся в допустимом начальном состоянии и согласованный с первыми i наблюдениями трассы, который оканчивается в состоянии x.

Доказательство. Доказательство проводится по индукции по i.

База индукции. Пусть $i = 0$. Тогда по определению 4.1 $D_0 = X_0$. Следовательно, для любого $x \in X$ условие $x \in D_0$ эквивалентно условию $x \in X_0$. Это, в свою очередь, эквивалентно существованию последовательности длины 1, состоящей из единственного состояния (x) и удовлетворяющей условиям утверждения. База доказана.

Переход индукции. Пусть утверждение верно для некоторого $i - 1 \in \{0, \ldots, m-1\}$. Докажем его для индекса i.

Необходимость. Пусть $x \in D_i$. Тогда по определению 4.2

$$\exists y \in D_{i-1} : C(\sigma_i, y, x) = 1.$$

По предположению индукции из условия $y \in D_{i-1}$ следует существование последовательности состояний $(x_1, x_1, \ldots, x_{i-1}) \in X^i$, для которой выполнены условия

$$x_0 \in X_0, \qquad x_{i-1} = y, \qquad \forall k \in \{1, \ldots, i-1\} : C(\sigma_k, x_{k-1}, x_k) = 1.$$

Добавляя к этой последовательности состояние x, получаем последовательность $(x_2, x_1, \ldots, x_{i-1}, x) \in X^{i+1}$, для которой дополнительно выполнено

$$C(\sigma_i, x_{i-1}, x) = 1.$$

Следовательно, существует последовательность состояний, удовлетворяющая условиям пункта б.

Достаточность. Пусть существует последовательность $(x_1, x_1, \ldots, x_i) \in X^{i+1}$, для которой выполнены условия

$$x_0 \in X_0, \qquad x_i = x, \qquad \forall k \in \{1, \ldots, i\} : C(\sigma_k, x_{k-1}, x_k) = 1.$$

Тогда её префикс $(x_1, x_1, \ldots, x_{i-1})$ удовлетворяет условиям для индекса $i-1$, а потому по предположению индукции $x_{i-1} \in D_{i-1}$. Из условия $C(\sigma_i, x_{i0}, x_i) = 1$ и определения 4.2 получаем $x_i \in D_i$. Так как $x_i = x$, то $x \in D_i$.

Тем самым обе импликации доказаны, следовательно, утверждения 1 и 2 эквивалентны. По принципу математической индукции утверждение доказано для всех $i \in \{0, \ldots, m\}$. □

4.4 Критерий существования согласованного поведения

Из доказанного индуктивного инварианта непосредственно вытекает основной критерий решения поставленной задачи.

Теорема 4.1. Трасса программы $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m)$ согласована с моделью $\mathcal{M}$ тогда и только тогда, когда

$$D_m \neq \varnothing. \tag{12}$$

Доказательство. По определению согласованности трассы с моделью требуется выполнение условия $\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) \neq \varnothing$. Это означает, что существует последовательность состояний

$$(x_0, x_1, \ldots, x_m) \in X^{m+1},$$

для которой выполнены условия

$$x_0 \in X_0, \qquad \forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1.$$

Согласно предыдущему утверждению, существование такой последовательности эквивалентно существованию состояния

$$x_m \in D_m.$$

Последнее, в свою очередь, эквивалентно условию $D_m \neq \varnothing$. □

Теорема 4.1 показывает, что задача разрешимости, сформулированная в предыдущем разделе, сводится к проверке непустоты множества D_m, построенного по рекуррентному правилу 4.2. Тем самым задача имеет не только абстрактную формулировку, но и конструктивное решение.

4.5 Алгоритм решения

Из построения множеств D_i непосредственно следует алгоритм решения задачи, приведенный в Алгоритме 1.

Алгоритм 1 — Проверка согласованности трассы с моделью

Исходные параметры: система переходов $\mathcal{M} = (X, X_0, R)$; трасса $\tau = (\sigma_1, \ldots, \sigma_m) \in \Sigma^m$; предикат согласованности $C : \Sigma \times X \times X \to \{0, 1\}$

Результат: ответ о согласованности трассы τ с моделью $\mathcal{M}$

```
D_0 ← X_0;
для каждого i ∈ {1, ..., m} выполнять
    D_i ← {y ∈ X : ∃x ∈ D_{i−1} : C(σ_i, x, y) = 1};
    если D_i = ∅ тогда
        возвратить τ не согласована с M;
    конец
конец
возвратить τ согласована с M;
```

Данный алгоритм является корректным в силу теоремы 4.1. Если множество состояний X конечно, то алгоритм является конструктивно осуществимым, так как каждое множество D_i вычисляется за конечное число шагов. В случае бесконечного пространства состояний для его реализации требуется дополнительный механизм перебора или символьного представления множеств состояний и отношений переходов, однако сама математическая редукция задачи при этом не изменяется.

Особый интерес представляет случай, когда $D_m = \varnothing$. Тогда трасса не согласована с моделью. Однако это не означает, что не существует ни одного согласованного префикса трассы. Напротив, может существовать максимальный индекс

$$k = \max \{ i \in \{0, \ldots, m\} : D_i \neq \varnothing \}, \tag{13}$$

после которого дальнейшее продолжение становится невозможным.

Смысл числа k состоит в том, что первые k наблюдений трассы ещё могут быть объяснены некоторым поведением модели, тогда как наблюдение σ_{k+1} уже не может быть согласовано ни с одним продолжением. Тем самым индекс k указывает на первый шаг трассы, на котором возникает расхождение между реализацией и моделью.

Это наблюдение важно не только теоретически, но и практически, поскольку позволяет использовать предложенный метод не только для принятия или отклонения трассы в целом, но и для локализации момента возникновения несоответствия.

5 Реализация задачи и решения средствами TLA+ и TLC

В предыдущих разделах задача верификации соответствия трассы программы и формальной спецификации была сформулирована как задача разрешимости: по заданным системе переходов $\mathcal{M} = (X, X_0, R)$, трассе наблюдений $\tau = (\sigma_1, \ldots, \sigma_m)$ и предикату согласованности шага $C : \Sigma \times X \times X \to \{0, 1\}$ требуется определить, существует ли конечное поведение модели, согласованное с трассой, то есть выполнено ли условие $\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) \neq \varnothing$.

В настоящем разделе показывается, каким образом эта математическая постановка реализуется средствами языка TLA+ и TLC [3] на примере спецификации протокола двухфазной транзакции.

5.1 Общая схема реализации

Практическая реализация предложенного метода включает следующее:

а) **Спецификация алгоритма.** Она задаёт исходную формальную модель проверяемой системы: множество переменных состояния, начальное условие и отношение переходов [4].

б) **Спецификация трассы.** Она строится поверх базовой спецификации и добавляет ограничения, порождаемые наблюдаемой трассой выполнения программы [4].

в) **Конфигурация проверки TLC.** Она задаёт конкретные значения констант, подключает нужную спецификацию и определяет условия, при которых трасса считается принятой или отвергнутой [4].

г) **Библиотека для трассировки программ.** Она позволяет формировать трассы выполнения программы и объединять их в единую трассу при необходимости [5].

5.2 Соответствие модели и TLA+ спецификации

С точки зрения математической постановки модель задаётся системой переходов

$$\mathcal{M} = (X, X_0, R),$$

где X — множество состояний, X_0 — множество начальных состояний, а R — отношение переходов.

```
CONSTANT RM
VARIABLES rmState, tmState, tmPrepared, msgs
vars == << rmState, tmState, tmPrepared, msgs >>
Messages == [type : {"Prepared"}, rm : RM] \cup [type : {"Commit", "Abort"}]

TPInit ==
    /\ rmState = [r \in RM ⟼ "working"] /\ tmState = "init"
    /\ tmPrepared = {} /\ msgs = {}

RMPrepare(r) ==
    /\ rmState[r] = "working"
    /\ rmState' = [rmState EXCEPT ![r] = "prepared"]
    /\ msgs' = msgs \cup { [type ⟼ "Prepared", rm ⟼ r] }

RMRcvCommitMsg(r) ==
  /\ [type ⟼ "Commit"] \in msgs
  /\ rmState' = [rmState EXCEPT ![r] = "committed"]

TMRcvPrepared(r) ==
    /\ tmState = "init"
    /\ [type ⟼ "Prepared", rm ⟼ r] \in msgs
    /\ tmPrepared' = tmPrepared \cup {r}

TMCommit ==
    /\ tmState = "init" /\ tmPrepared = RM
    /\ tmState' = "done" /\ msgs' = msgs \cup { [type ⟼ "Commit"] }

(* RMRcvAbortMsg == ... *)
(* TMAbort == ... *)

TPNext ==
    \/ TMCommit \/ TMAbort
    \/ \E r \in RM :
         \/ RMPrepare(r)
         \/ TMRcvPrepared(r)

Spec == TPInit /\ [][TPNext]_vars

Consistent ==
    \A r1, r2 \in RM : ~(rmState[r1] = "aborted" /\ rmState[r2] = "committed")
```

Рисунок 1 – Спецификация алгоритма двухфазной транзакции

Для модуля `TwoPhase` (см. рис. 6) вектор состояния задаётся переменными

$$\texttt{vars} = \langle \texttt{rmState}, \texttt{tmState}, \texttt{tmPrepared}, \texttt{msgs} \rangle.$$

Следовательно, множество X в данном примере можно интерпретировать как множество всех допустимых значений указанной четвёрки переменных. Начальному множеству X_0 соответствует предикат `TPInit`, а отношению переходов R соответствует действие `TPNext`. Полная спецификация `Spec` задаёт

множество допустимых бесконечных поведений, однако для цели валидации трасс существенны только конечные префиксы таких поведений, то есть именно те объекты, которые в математической постановке были обозначены через $\mathrm{B}(\mathcal{M})$.

Таким образом, исходная спецификация алгоритма TLA+ задает те самые переходные системы, которые были введены в математической части.

5.3 Представление трассы программы

В математической постановке трасса программы задавалась последовательностью наблюдений $\tau = (\sigma_1, \sigma_2, \ldots, \sigma_m) \in \Sigma^m$. В рассматриваемой реализации эта последовательность хранится во внешнем файле в формате NDJSON и считывается оператором `Trace` (см. рис. **??**).

```
1 Trace ==
2     IF "TRACE_PATH" \in DOMAIN IOEnv THEN
3         ndJsonDeserialize(IOEnv.TRACE_PATH)
4     ELSE
5         Print(<<"TRACE_PATH environnement variable not found">>)
```

Рисунок 2 – Счтывание трассы программы

Каждая строка такого файла представляет собой запись, описывающую один шаг исполнения программы. Запись может содержать имя события, его параметры и набор частичных обновлений логических переменных модели, пример такой записи представлен на рис 3.

```
1 {
2   "clock": 4,
3   "rmState": [
4     { "op": "Update",
5       "path": ["rm-0"],
6       "args": ["prepared"] }
7   ],
8   "msgs": [
9     { "op": "AddElement",
10      "path": [],
11      "args": [
12        { "type":"Prepared", "rm":"rm-0" }
13      ]
14    }
15  ],
16  "event": "RMPrepare",
17  "event_args": ["rm-0"]
18 }
```

Рисунок 3 – Пример наблюдения трассы

Тем самым множество наблюдений Σ в реализации соответствует множеству всех допустимых записей трассы данного формата. Последовательность `Trace` и есть конкретная реализация трассы τ.

5.4 Реализация предиката согласованности шага

Существенным свойством рассматриваемой реализации является синхронность сопоставления трассы и поведения модели. В математической постановке это выражено требованием, что каждой записи трассы соответствует ровно один переход согласованного поведения модели. В реализации это обеспечивается переменной `l` и оператором `IsEvent`, реализация которого представлена на рис. 4.

```
1 IsEvent(e) ==
2     \* Equals FALSE if we get past the end of the log, causing checking to stop.
3     /\ l \in 1..Len(Trace)
4     /\ IF "event" \in DOMAIN logline THEN logline.event = e ELSE TRUE
5     /\ l' = l + 1
6     /\ UpdateVariables(logline)
```

Рисунок 4 – Реализация `IsEvent`

Оператор `IsEvent(e)` содержит следующие ограничения:

а) текущий индекс должен принадлежать диапазону `l` $\in 1..Len($`Trace`$)$;

б) имя события в текущей записи, если оно задано, должно совпадать с ожидаемым событием `e`;

в) после обработки текущей записи индекс увеличивается: `l'` $=$ `l` $+ 1$.

Следовательно, каждая успешно обработанная запись трассы продвигает систему на один шаг вперёд. Это и есть программная реализация условия

$$\forall i \in \{1, \ldots, m\} : C(\sigma_i, x_{i-1}, x_i) = 1$$

из математической постановки. Иными словами, в текущем подходе одна запись трассы соответствует одному переходу ограниченной спецификации. Следует подчеркнуть, что речь идёт именно о переходе, а не о состоянии. Запись трассы интерпретируется как ограничение на пару соседних состояний модели.

В математической части предикат $C(\sigma, x, y)$ был введён абстрактно: он должен принимать значение 1 тогда и только тогда, когда наблюдение σ согласовано с переходом из состояния x в состояние y. В TLA+-реализации этот предикат не задаётся отдельной формулой, а реализуется через сочетание трёх механизмов (см. рис. 5:

а) проверки имени события;

б) проверки параметров события;

в) проверки и вычисления значений штрихованных переменных.

```
1  -------------------------- MODULE TwoPhaseTrace --------------------------
2  (***************************************************************************)
3  (* Trace spec that refines 2PC *)
4  (***************************************************************************)
5
6  EXTENDS TLC, Sequences, SequencesExt, Naturals, FiniteSets, Bags, Json,
7          IOUtils, TwoPhase, TVOperators, TraceSpec
8
9  (* Can be extracted from init *)
10 TPDefault(varName) ==
11     CASE varName = "rmState" -> [r \in RM |-> "working"]
12     []  varName = "tmState" -> "init"
13     []  varName = "tmPrepared" -> {}
14     []  varName = "msgs" -> {}
15
16 TPUpdateVariables(t) ==
17     /\
18         IF "rmState" \in DOMAIN t
19         THEN rmState' = UpdateVariable(rmState, "rmState", t)
20         ELSE TRUE
21     (*...*)
22
23 (* Predicate actions *)
24 IsTMCommit ==
25     /\ IsEvent("TMCommit")
26     /\ TMCommit
27
28 (* ... *)
29
30 TPTraceNext ==
31         \/ IsTMCommit
32         (* ... *)
33 ===========================================================================
```

Рисунок 5 – Обертка над спецификацией алгоритма для верификации трасс

Проверка имени события реализуется оператором `IsEvent`, описание которого приведено выше. Проверка параметров реализуется в действиях вида `IsTMCommit`, `IsRMPrepare`, `IsRMRcvCommitMsg` и других: если параметры заданы в трассе, то выбирается соответствующий экземпляр действия, а

если не заданы, то допускается выбор любого допустимого параметра. Тем самым частичная информация, присутствующая в трассе, корректно интерпретируется как ограничение на множество допустимых переходов модели.

Проверка обновлений переменных реализуется оператором `TPUpdateVariables`. Он анализирует текущую запись трассы и, если для какой-либо переменной задан список обновлений, вычисляет соответствующее значение переменной при помощи общего оператора `ApplyUpdates`. Модуль `TVOperators` содержит универсальные операции `Update`, `AddElement`, `RemoveElement`, `SetKey` и другие, что позволяет интерпретировать записи трассы как частичные преобразования сложных структур данных.

Например, если запись трассы фиксирует, что в множество сообщений был добавлен элемент типа `Prepared`, а состояние некоторого ресурсного менеджера изменилось на `"prepared"`, то `TPUpdateVariables` накладывает ровно эти ограничения на значения `msgs'` и `rmState'`. Совместно с действием `RMPrepare(r)` это даёт точную реализацию условия согласованности данного наблюдения с переходом модели.

Таким образом, конкретная реализация предиката C зависит от выбранной спецификации и формата трассы, но общая схема остаётся неизменной: наблюдение задаёт ограничения на разрешённый шаг модели, а TLC проверяет, существует ли переход, удовлетворяющий этим ограничениям.

5.5 Ограниченная спецификация как реализация $\mathrm{M}(\tau)$

В математической постановке множество $\mathrm{M}(\tau)$ было определено как множество всех последовательностей состояний, согласованных с трассой. Средствами TLA+ это множество реализуется неявно через построение *ограниченной спецификации*, которая разрешает только те шаги, которые совместимы с текущей записью трассы.

В модуле `TraceSpec` такая ограниченная спецификация задаётся формулой

$$\mathtt{TraceSpec} \equiv \mathtt{TraceInit} \wedge [\,][\mathtt{TraceNext}]_{\langle \mathtt{l}, \mathtt{Vars} \rangle}.$$

Здесь `Vars` обозначает вектор переменных исходной спецификации, `BaseInit` — её начальный предикат, а `TraceNext` — дизъюнкция ограниченных действий, соответствующих шагам исходной модели.

Каждое из действий `TraceNext` соответствует некоторому действию исходной спецификации, но дополнительно ограничено текущей записью трассы. Следовательно, поведения ограниченной спецификации представляют собой ровно те поведения исходной модели, которые согласованы с данной трассой. Иначе говоря, ограниченная спецификация реализует множество

$$\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau).$$

С практической точки зрения это означает следующее: TLC выполняет обычный обход пространства состояний, но не всей исходной спецификации, а только той её части, которая совместима с наблюдаемой трассой. Если соответствующее пространство состояний непусто, то трасса согласована с моделью; если оно пусто, то согласованного поведения не существует.

5.6 Соответствие рекуррентных множеств D_i и работы TLC

В математическом обосновании решения были введены множества

$$D_0, D_1, \ldots, D_m,$$

где D_i интерпретировалось как множество всех состояний модели, достижимых после согласования первых i наблюдений трассы. В реализации на TLC эти множества не вычисляются явно как отдельные математические объекты, однако вся процедура обхода ограниченной спецификации фактически эквивалентна именно такому вычислению.

Действительно, переменная `l` хранит номер текущей записи трассы. Начальное состояние ограниченной спецификации соответствует значению $\mathtt{l} = 1$, то есть ещё не обработано ни одной записи трассы. После обработки первой записи получаются состояния со значением $\mathtt{l} = 2$, после обработки второй — со значением $\mathtt{l} = 3$, и так далее. Поэтому множество всех достижимых состояний ограниченной спецификации с фиксированным значением

$$\mathtt{l} = i + 1$$

естественно соответствует множеству D_i из математической постановки.

Иначе говоря, TLC реализует пошаговое построение множеств D_i не в явном формульном виде, а через стандартный перебор достижимых состояний ограниченной спецификации. Это и есть алгоритмическая интерпретация доказанного в предыдущем разделе критерия $D_m \neq \varnothing$.

5.7 Критерий успешной проверки в TLC

В математической постановке критерий согласованности был выражен условием $\mathrm{B}(\mathcal{M}) \cap \mathrm{M}(\tau) \neq \varnothing$, а в разделе обоснования решения было показано, что этот критерий эквивалентен условию $D_m \neq \varnothing$. В реализации на TLC данная идея выражается через постусловие `TraceAccepted`.

```
CONSTANT RM
VARIABLES rmState, tmState, tmPrepared, msgs
vars == << rmState, tmState, tmPrepared, msgs >>
Messages == [type : {"Prepared"}, rm : RM] \cup [type : {"Commit", "Abort"}]

TPInit ==
    /\ rmState = [r \in RM ⟼ "working"] /\ tmState = "init"
    /\ tmPrepared = {} /\ msgs = {}

RMPrepare(r) ==
    /\ rmState[r] = "working"
    /\ rmState' = [rmState EXCEPT ![r] = "prepared"]
    /\ msgs' = msgs \cup { [type ⟼ "Prepared", rm ⟼ r] }

RMRcvCommitMsg(r) ==
  /\ [type ⟼ "Commit"] \in msgs
  /\ rmState' = [rmState EXCEPT ![r] = "committed"]

TMRcvPrepared(r) ==
    /\ tmState = "init"
    /\ [type ⟼ "Prepared", rm ⟼ r] \in msgs
    /\ tmPrepared' = tmPrepared \cup {r}

TMCommit ==
    /\ tmState = "init" /\ tmPrepared = RM
    /\ tmState' = "done" /\ msgs' = msgs \cup { [type ⟼ "Commit"] }

(* RMRcvAbortMsg == ... *)
(* TMAbort == ... *)

TPNext ==
    \/ TMCommit \/ TMAbort
    \/ \E r \in RM :
         \/ RMPrepare(r)
         \/ TMRcvPrepared(r)

Spec == TPInit /\ [][TPNext]_vars

Consistent ==
    \A r1, r2 \in RM : ~(rmState[r1] = "aborted" /\ rmState[r2] = "committed")
```

Рисунок 6 – Реализация `TraceAccepted`

Оператор `TraceAccepted` использует статистику TLC и проверяет, что диаметр построенного ограниченного пространства состояний совпадает с длиной трассы с точностью до единицы. Содержательно это означает, что TLC удалось построить путь, последовательно согласующий все записи трассы. Если же такой путь отсутствует, то значение `TraceAccepted` ложно, и проверка завершается сообщением об ошибке.

Использование диаметра здесь оправдано структурой ограниченной спецификации: при каждом шаге переменная `l` увеличивается на единицу, поэтому обход пространства состояний естественным образом организован по уровням, соответствующим длине уже согласованного префикса трассы. Следовательно, достижение последнего уровня эквивалентно существованию согласованного поведения полной длины.

5.8 Особенности и ограничения рассматриваемой реализации

Следует отметить, что реализованная схема соответствует той математической постановке, которая была принята в работе, а именно синхронному сопоставлению трассы и поведения модели. Каждой записи трассы соответствует ровно один переход ограниченной спецификации. Это упрощает как математическое описание, так и программную реализацию.

Вместе с тем структура модулей показывает, что возможны и более общие расширения. В частности, в коде предусмотрены заготовки для обработки stuttering-переходов и для задания композиции действий, однако в текущем варианте они отключены. Следовательно, реализованное решение охватывает тот класс задач, в котором инструментированная трасса уже приведена к уровню атомарности, совместимому с шагами спецификации.

Ещё одно ограничение связано с неполнотой трассы. Если записи трассы содержат слишком мало информации, то множество допустимых интерпретаций отдельного шага становится слишком широким. Тогда TLC может найти некоторое согласованное поведение модели, хотя фактическое поведение программы восстановлено по трассе неоднозначно. Таким образом, практическая точность метода зависит не только от корректности спецификации, но и от качества инструментирования кода и полноты логируемых данных.

ЗАКЛЮЧЕНИЕ

В работе рассмотрена задача верификации соответствия трасс исполнения программы формальной спецификации, заданной на языке TLA+. Показано, что данная задача возникает при необходимости поддерживать согласованность между реализацией и формальной моделью в процессе эволюции программной системы.

В ходе работы была предложена методика верификации, основанная на инструментировании программной реализации, построении трассы наблюдаемых шагов исполнения и её последующей интерпретации как системы ограничений на переходы формальной модели. Такой подход позволяет сопоставлять поведение программы со спецификацией не по исходному коду напрямую, а по наблюдаемым эффектам выполнения, выраженным в терминах логических переменных модели.

Основным теоретическим результатом работы стала строгая математическая постановка задачи верификации в терминах системы переходов, множества допустимых поведений модели, множества поведений, согласованных с трассой, и предиката согласованности шага. Показано, что рассматриваемая задача является задачей разрешимости и сводится к проверке непустоты пересечения множеств допустимых поведений модели и поведений, согласованных с наблюдаемой трассой.

Для решения поставленной задачи введена последовательность множеств достижимых состояний, согласованных с префиксами трассы. Доказано, что каждое такое множество содержит в точности те состояния модели, которые достижимы после согласования соответствующего префикса наблюдений. На этой основе получен критерий существования согласованного поведения и построен алгоритм проверки согласованности трассы с моделью.

Практическая часть работы показала, каким образом полученная математическая постановка реализуется средствами TLA+ и TLC. Рассмотрено соответствие между переходной системой и TLA+-спецификацией, между наблюдаемой трассой и ограниченной спецификацией, а также между рекуррентным построением множеств достижимых состояний и обходом пространства состояний, выполняемым TLC. Показано, что предложенная схема может быть реализована на примере спецификации протокола двухфазной фиксации транзакции.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ПОЛОЖЕНИЕ №03.02-06/2. О структурном подразделении Научно-учебный комплекс «Информатика и системы управления». — Взамен Положения о структурном подразделении №03.02-06/1 от 12.04.2012. — МГТУ им. Н.Э. Баумана, 2022. — 8 с.

2. Keller R. M. Formal Verification of Parallel Programs // Communications of the ACM. — 1976. — Июль. — Т. 19, № 7. — С. 371—384.

3. Lamport L. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. — Boston, MA : Addison-Wesley, 2002.

4. Железцов Н. two-phase-tlac: реализации алгоритма двухфазного коммита для валидации соответствия TLA+ спецификации. — 2025. — URL: https://github.com/Serpentian/two-phase-tlac (дата обращения 15.11.2025).

5. Железцов Н. tlac: C библиотека для трассировки кода. — 2025. — URL: https://github.com/Serpentian/tlac (дата обращения 15.09.2025).

ПРИЛОЖЕНИЕ А

Методика верификации

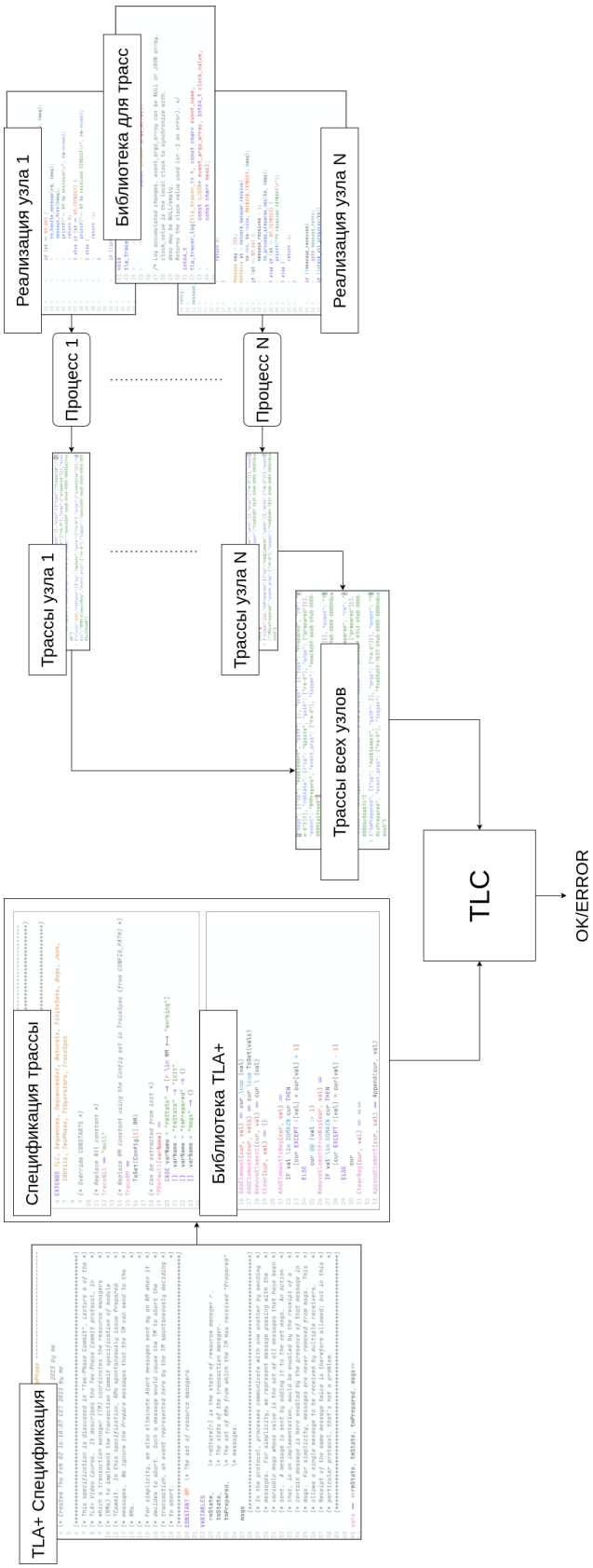

Рисунок А.1 – Методика верификации